

Matplotlib – 13 Practical Exercises

Beginner-friendly practice workbook covering bar charts, grouped bar charts, scatter plots, histograms, pie charts, box plots, and outlier detection.

How to Use This PDF

Run each program in Jupyter Notebook or VS Code. First understand the data, then type the code yourself, change a few values, and observe how the chart changes.

1. Bar Chart – Sales for Five Products

```
import matplotlib.pyplot as plt

products = ["Laptop", "Mobile", "Tablet", "Watch", "Headphones"]
sales = [500, 800, 300, 450, 600]

plt.bar(products, sales)

plt.xlabel("Products")
plt.ylabel("Sales")
plt.title("Product Sales")

plt.show()
```

Why this chart? A bar chart is appropriate because it makes it easy to compare sales across different product categories.

Explanation: Key idea: `plt.bar()` creates vertical bars. The first list supplies categories and the second list supplies bar heights.

2. Horizontal Bar Chart

```
import matplotlib.pyplot as plt

products = ["Laptop", "Mobile", "Tablet", "Watch", "Headphones"]
sales = [500, 800, 300, 450, 600]

plt.barh(products, sales)

plt.xlabel("Sales")
plt.ylabel("Products")
plt.title("Product Sales - Horizontal")

plt.show()
```

Why this chart? A horizontal bar chart is useful when category names are long because the labels are easier to read.

Explanation: Key idea: `plt.barh()` creates horizontal bars instead of vertical bars.

3. Grouped Bar Chart – Two Years

```
import numpy as np
import matplotlib.pyplot as plt

products = ["Laptop", "Mobile", "Tablet", "Watch", "Headphones"]
sales_2025 = [500, 800, 300, 450, 600]
sales_2026 = [650, 900, 420, 550, 750]

x = np.arange(len(products))
width = 0.35
```

```
plt.bar(x - width/2, sales_2025, width, label="2025")
plt.bar(x + width/2, sales_2026, width, label="2026")

plt.xticks(x, products)
plt.xlabel("Products")
plt.ylabel("Sales")
plt.title("Sales Comparison")
plt.legend()

plt.show()
```

Why this chart? A grouped bar chart is appropriate because it allows direct comparison of two years of sales for each product.

Explanation: Key idea: $x - \text{width}/2$ moves the first bar left and $x + \text{width}/2$ moves the second bar right so the bars appear side by side.

4. Add Numerical Values Above Bars

```
import matplotlib.pyplot as plt

products = ["Laptop", "Mobile", "Tablet", "Watch", "Headphones"]
sales = [500, 800, 300, 450, 600]

bars = plt.bar(products, sales)

plt.bar_label(bars, padding=3)

plt.xlabel("Products")
plt.ylabel("Sales")
plt.title("Product Sales with Values")

plt.show()
```

Why this chart? Adding values makes exact sales figures visible without requiring the viewer to estimate them from the y-axis.

Explanation: Key idea: `plt.bar_label()` adds the numerical value of each bar.

5. Scatter Plot – Advertising Spend vs Sales

```
import matplotlib.pyplot as plt

advertising = [10, 20, 30, 40, 50, 60, 70, 80]
sales = [100, 150, 180, 250, 300, 350, 400, 480]

plt.scatter(advertising, sales)

plt.xlabel("Advertising Spend (■ thousands)")
plt.ylabel("Sales (■ thousands)")
plt.title("Advertising vs Sales")

plt.show()
```

Why this chart? A scatter plot is appropriate because it helps visualize the relationship between advertising expenditure and sales.

Explanation: Key idea: each dot represents one observation. A visible upward pattern indicates a positive association in this example, but does not by itself prove causation.

6. Scatter Plot – Study Hours vs Exam Score

```
import matplotlib.pyplot as plt

study_hours = [1, 2, 3, 4, 5, 6, 7, 8]
```

```

exam_scores = [35, 40, 50, 55, 65, 70, 80, 90]

plt.scatter(study_hours, exam_scores)

plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score")

plt.show()

```

Why this chart? A scatter plot is suitable for examining whether a relationship exists between study hours and exam scores.

Explanation: Key idea: put the possible input/independent variable on the x-axis and the measured outcome on the y-axis.

7. Histogram – Customer Ages, 5 and 10 Bins

```

import matplotlib.pyplot as plt

ages = [
    18, 20, 22, 25, 27, 29, 30, 32,
    34, 35, 36, 38, 40, 42, 44, 45,
    47, 49, 50, 52, 55, 57, 60, 62,
    65, 68, 70, 72, 75, 78
]

# 5 bins
plt.hist(ages, bins=5, edgecolor="black")
plt.xlabel("Customer Age")
plt.ylabel("Frequency")
plt.title("Customer Age Distribution (5 bins)")
plt.show()

# 10 bins
plt.hist(ages, bins=10, edgecolor="black")
plt.xlabel("Customer Age")
plt.ylabel("Frequency")
plt.title("Customer Age Distribution (10 bins)")
plt.show()

```

Why this chart? A histogram is appropriate because customer age is numerical data and a histogram shows how frequently ages occur within intervals.

Explanation: Key idea: bins divide the numerical range into intervals. More bins generally mean narrower intervals.

8. How Changing the Number of Bins Changes Interpretation

```

# Try different numbers of bins

plt.hist(ages, bins=5, edgecolor="black")
plt.title("5 Bins")
plt.show()

plt.hist(ages, bins=10, edgecolor="black")
plt.title("10 Bins")
plt.show()

plt.hist(ages, bins=20, edgecolor="black")
plt.title("20 Bins")
plt.show()

```

Why this chart? Comparing bin counts is useful because it shows how grouping numerical data can change the apparent level of detail without changing the underlying observations.

Explanation: 5 bins → broader, simpler view. 10 bins → more detail. 20 bins → even finer detail, but potentially more visual noise. Too few bins can hide patterns; too many can exaggerate random fluctuations.

9. Pie Chart – Customer Type Distribution

```
import matplotlib.pyplot as plt

customer_types = ["New", "Returning", "Premium"]
customers = [400, 350, 250]

plt.pie(
    customers,
    labels=customer_types,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Customer Type Distribution")
plt.show()
```

Why this chart? A pie chart is appropriate for showing how each customer category contributes to the whole when there are only a few categories.

Explanation: Key idea: autopct displays percentages and startangle rotates the starting position.

10. Box Plot – Employee Salaries

```
import matplotlib.pyplot as plt

salaries = [
    25000, 28000, 30000, 32000, 35000,
    38000, 40000, 42000, 45000, 50000,
    55000, 60000, 65000, 70000, 150000
]

plt.boxplot(salaries)

plt.ylabel("Salary (■)")
plt.title("Employee Salary Distribution")

plt.show()
```

Why this chart? A box plot is appropriate because it summarizes salary distribution and helps reveal spread, median, quartiles, and potential outliers.

Explanation: Key idea: a box plot shows the median, Q1, Q3, whiskers, and possible outlying observations.

11. Side-by-Side Box Plots – Salary by Department

```
import matplotlib.pyplot as plt

it = [35000, 40000, 45000, 50000, 55000, 60000]
hr = [25000, 28000, 30000, 32000, 35000, 38000]
sales = [30000, 35000, 40000, 45000, 50000, 90000]

plt.boxplot(
    [it, hr, sales],
    tick_labels=["IT", "HR", "Sales"]
)

plt.xlabel("Department")
plt.ylabel("Salary (■)")
plt.title("Salary by Department")

plt.show()
```

Why this chart? Side-by-side box plots are appropriate because they let you compare salary distributions, medians, and variability across departments on a common scale.

Explanation: Key idea: each department gets its own box, making differences in central tendency and spread easier to compare.

12. Identify Potential Outliers Using IQR

```
import numpy as np
import matplotlib.pyplot as plt

salaries = [
    25000, 28000, 30000, 32000, 35000,
    38000, 40000, 42000, 45000, 50000,
    55000, 60000, 65000, 70000, 150000
]

Q1 = np.percentile(salaries, 25)
Q3 = np.percentile(salaries, 75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = [
    x for x in salaries
    if x < lower or x > upper
]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower bound:", lower)
print("Upper bound:", upper)
print("Outliers:", outliers)

plt.boxplot(salaries)
plt.title("Salary Outlier Detection")
plt.ylabel("Salary (■)")
plt.show()
```

Why this chart? A box plot combined with the IQR rule is appropriate because it provides a standard way to flag observations that are unusually far from the central spread.

Explanation: Formula: $IQR = Q3 - Q1$. Lower bound = $Q1 - 1.5 \times IQR$. Upper bound = $Q3 + 1.5 \times IQR$. Values outside these bounds are potential outliers, not automatically errors.

13. Chart Selection – Quick Revision

Why this chart? Choosing a chart based on the question helps communicate data clearly.

Explanation: Bar → compare categories. Horizontal bar → compare categories with long labels. Grouped bar → compare categories across groups/time. Scatter → examine relationships between two numerical variables. Histogram → examine a numerical distribution. Pie → show parts of a whole for a small number of categories. Box plot → compare distributions and identify potential outliers.

Matplotlib Chart Selection Cheat Sheet

Task	Chart	Main Purpose
Product sales	Bar	Compare categories
Horizontal sales	Barh	Compare categories / long labels
Two-year sales	Grouped bar	Compare values side by side
Exact bar values	Bar + bar_label	Show numerical values
Advertising vs sales	Scatter	Explore relationship
Study hours vs score	Scatter	Explore association
Customer ages	Histogram	Show numerical distribution
Customer types	Pie	Show parts of a whole
Employee salaries	Box plot	Summarize spread
Salary by department	Box plots	Compare distributions
Potential outliers	Box plot + IQR	Flag unusual observations

Important Matplotlib Commands

<code>plt.bar()</code>	→ Vertical bar chart
<code>plt.barh()</code>	→ Horizontal bar chart
<code>plt.scatter()</code>	→ Scatter plot
<code>plt.hist()</code>	→ Histogram
<code>plt.pie()</code>	→ Pie chart
<code>plt.boxplot()</code>	→ Box plot
<code>plt.xlabel()</code>	→ X-axis label
<code>plt.ylabel()</code>	→ Y-axis label
<code>plt.title()</code>	→ Chart title
<code>plt.legend()</code>	→ Display legend
<code>plt.xticks()</code>	→ Change X-axis tick labels
<code>plt.bar_label()</code>	→ Add values to bars
<code>plt.show()</code>	→ Display chart

Practice Rule

Do not just copy the code. Change the products, sales, salaries, ages, and other values. Then predict what the chart should look like before running it. This is the best way to build practical Matplotlib understanding.